

Dissertation on

"Adaptive Study Planning for Self-Directed Learners"

Submitted in partial fulfilment of the requirements for the award of degree of

Master of Technology in
Data Science and Artificial Intelligence

UE20CS972 Project Phase - 2

Submitted by:
Rohit Saji
PES2PGE24DS201

Under the guidance of
Prof. Ramesh Prakash Guledgudd
PES University

May – October 2026

M.TECH IN DATA SCIENCE AND ARTIFICIAL INTELLIGENCE
FACULTY OF ENGINEERING
PES UNIVERSITY

(Established under Karnataka Act No. 16 of 2013)
Electronic City, Hosur Road, Bengaluru – 560 100, Karnataka, India

PES UNIVERSITY
(Established under Karnataka Act No. 16 of 2013)
Electronic City, Hosur Road, Bengaluru – 560 100, Karnataka, India

FACULTY OF ENGINEERING

CERTIFICATE

This is to certify that the dissertation entitled "Adaptive Study Planning for Self-Directed Learners" is a bonafide work carried out by **Rohit Saji (PES2PGE24DS201)** in partial fulfilment for the completion of Project Phase - 2 (UE20CS972) in the Program of Study – Master of Technology in Data Science and Artificial Intelligence under the rules and regulations of PES University, Bengaluru during the period May 2026 – October 2026. It is certified that all corrections/suggestions indicated for internal assessment have been incorporated in the report. The dissertation has been approved as it satisfies the academic requirements in respect of project work.

External Viva

Name of the Examiners	Signature with Date
1. _____	_____
2. _____	_____

Signature	Signature	Signature
Prof. Ramesh Prakash Guledgudd	Dr. Sandesh B J	Dr. Keshavan B K
PES University	Chairperson	Dean of Faculty

Declaration

I hereby declare that the Project Phase - 2 entitled “Adaptive Study Planning for Self-Directed Learners” has been carried out under the guidance of Prof. Ramesh Prakash Guledgudd, and submitted in partial fulfilment of the course requirements for the award of the degree of Master of Technology in Data Science and Artificial Intelligence of PES University, Bengaluru, during the academic semester May – October 2026.

Acknowledgement

I would like to express my gratitude to my guide for continuous guidance and support throughout this project, and to the project coordinators for their help. I thank the Chairperson and faculty of the Department, PES University, for their support, and the university leadership for the opportunities provided. Finally, I extend heartfelt thanks to my family and friends for their constant encouragement.

Abstract

Self-directed learners routinely commit to a fixed study plan and fall behind or move ahead within days, yet the plan does not adapt and the learner has no reliable signal of how far off-track they are. Phase I of this work built an open-loop adaptive planner that estimates the learner's true study pace from session history, detects behavioural shifts in that pace, and projects the completion date with a calibrated uncertainty band, regenerating the plan when the projection diverges from the deadline. The open loop adapts to pace but never checks learning: logged hours do not prove that studying produced understanding. Phase II closes the loop with assessment-based verification of genuine learning. The system generates grounded assessments in three question families from the learner's own materials, grades each family with an authoritative arm, tracks mastery with Bayesian knowledge tracing, guides with a Socratic ladder and a gated reveal, returns advisory feedback copy, runs mixed practice through the library and practice hub, and replans from the loop verdict. Each component is evaluated against baselines and live runs, and the closed loop is validated on synthetic longitudinal histories across four learner archetypes. The selected models are realised in a working local-first web application.

Contents

1 Introduction **10**
- 1.1 Background . . . 10
- 1.2 Problem Statement . . . 10
- 1.3 Objectives . . . 11
- 1.4 Scope of the Project . . . 12
- 1.5 Organization of the Report . . . 12

2 Literature Survey **13**
- 2.1 Knowledge Tracing and Mastery . . . 13
- 2.2 Automatic Item Generation and KC Tagging . . . 14
- 2.3 LLM and Sandbox Grading . . . 14
- 2.4 Socratic Guidance and Feedback . . . 15
- 2.5 Methodology for Sourcing the Surveyed Work . . . 15
- 2.6 Summary of Approaches . . . 16

3 System Requirements Specification **18**
- 3.1 Hardware Requirements . . . 18
- 3.2 Software Requirements . . . 19
- 3.3 Functional Requirements . . . 19
- 3.4 Non-Functional Requirements . . . 20

4 Proposed Methodology **22**
- 4.1 The Closed Learning Loop . . . 22
- 4.2 Candidate Models and Selected Approach . . . 22
  - 4.2.1 Item Generation . . . 23
  - 4.2.2 Grading . . . 23
  - 4.2.3 Mastery and Difficulty . . . 23
  - 4.2.4 Guidance and Feedback . . . 23
  - 4.2.5 Loop Verdict . . . 23
- 4.3 Comparison Methodology . . . 24
- 4.4 System Design and Evolution . . . 25
- 4.5 System Architecture . . . 25

5 Implementation Details **27**
5.1 Development Environment . . . 27
5.2 Datasets and Their Sources . . . 27
5.3 Model and Service Implementation . . . 29
5.4 Evaluation Metrics and Reproducibility . . . 30
5.5 The Phase-2 Application . . . 30
5.5.1 Materials and Ingestion . . . 31
5.5.2 Assessment Generation . . . 31
5.5.3 Taking and Review . . . 33
5.5.4 Practice and Guidance . . . 33
5.5.5 Feedback and the Closed Loop . . . 34

6 Results and Discussion **36**
6.1 Generation Quality . . . 36
6.2 Live Verification . . . 38
6.3 Closed-Loop Decision . . . 38
6.4 Synthetic Longitudinal Validation . . . 39
6.5 Discussion . . . 39

7 Conclusion and Future Work **41**
7.1 Conclusion . . . 41
7.2 Future Work . . . 42

A Supplementary Tables and Figures **43**
A.1 Adherence Emission . . . 43
A.2 Verdict Instants . . . 43
A.3 Second-Corpus Questions and Endpoint Pointers . . . 44

List of Figures

2.1 The assessment-verified closed loop this chapter motivates: generated items are graded per family, grades update mastery, the verdict steers the plan, and the plan points back at the material. Each stage is surveyed in Sections 2.1 to 2.4. 16

4.1 The two-leg comparison. Fixed golds score each candidate arm behind a fail-closed gate; the synthetic replay then scores the whole loop before the approach is selected. 24
4.2 The deployment that realises Figure 2.1. The client derives views from the local log, the service fits models to history with the judge folded inside, Supabase hubs the data, and the sidecars serve on demand. 26

5.1 Application flow after Section 5.5.1: materials reach readiness in the library. 31
5.2 The material library: ready cards offer practice while the failed card keeps its badge with a retry action. 31
5.3 Application flow after Section 5.5.2: generation binds one material, one family, and one chapter per question. 32
5.4 Assessment generation: one material, one family, and one scoped chapter per question. 32
5.5 Application flow after Section 5.5.3: taking collects answers and review shows the server grade. 33
5.6 Taking and review: the server grade with per-criterion rubric feedback. 33
5.7 Application flow after Section 5.5.4: practice runs resolve lazily and the coach guides per attempt. 34
5.8 Practice run: the lazily resolved question with the Socratic coach anchored to the attempt. 34
5.9 Application flow after Section 5.5.5: the verdict closes the loop onto the replan. . . 35
5.10 Feedback and the loop: the inspectable evidence trail with the advisory recommendation. 35

List of Tables

2.1 Summary of Phase II approach families, with strengths and limitations for this project. . . . 16

3.1 Hardware environment. . . . 18
3.2 Software environment by tier. . . . 19

4.1 Candidate approaches per loop stage and the approach selected for the delivered system. . . . 24
4.2 Evaluation metrics by loop stage. . . . 25

5.1 Development environment: tools and libraries. The first eleven rows carry the Phase-I stack; the last six add the Phase-II loop. . . . 28
5.2 Datasets used in Phase II and their sources. . . . 28
5.3 Main packages and their roles. . . . 30

6.1 Retrieval: recall and MRR by arm. ACCA panels share the 30-question harness; the endpoint row is the live deployed path; DDIA is the second corpus. . . . 37
6.2 Generation: S-layer aggregate, groundedness judge, and coding pair. Judge and citation metrics are distinct by construction. . . . 37
6.3 Knowledge tracing: AUC and calibration across three populations. Framed as parity, not a win. . . . 38
6.4 Loop verdict: post-fix replay across four archetypes. Per-user sufficiency and decidedAt values sit in the versioned JSON beside the replay evidence. . . . 38

A.1 Adherence emission: actual over planned minutes by user and week. Week 5 is the post-replan extension week. . . . 43
A.2 Loop verdict instants: per-user decidedAt values. . . . 44
A.3 DDIA second-corpus gold set, questions 1 to 15. Snippets are verbatim from the evidence file. . . . 45
A.3 DDIA second-corpus gold set, questions 16 to 30. . . . 46

Chapter 1

Introduction

This chapter sets out the problem this phase addresses and the shape of the work that follows. Section 1.1 recalls the Phase I system in one paragraph. Section 1.2 states the Phase II problem. Section 1.3 lists the objectives of Phase II, each with a short explanation. Section 1.4 defines the scope of the work across both phases. Section 1.5 maps the chapters that follow.

1.1 Background

A self-directed learner is a person who works through online courses, examination preparation, or self-study material without an instructor setting the pace. Such a learner typically begins with a fixed plan, of the form "two hours a day, finish by this date." Within days the plan no longer matches reality, and the fixed finish date quietly turns wrong while the plan still displays it as though it were still valid. Phase I of this project built an open-loop adaptive planner for this learner. The system estimated true study pace from session history with hierarchical Bayesian calibration, flagged genuine shifts in pace with CUSUM change-point detection, projected the completion date with a calibrated uncertainty band using Gaussian-process regression, and regenerated the schedule from realised capacity when the projection diverged from the deadline. Each component was compared against baselines on a synthetic dataset with known ground truth, using held-out evaluation with bootstrap confidence intervals and multiple-comparison correction, and the selected models were realised in a working local-first web application. The comparison reported its nulls honestly: the calibrated pace beat the fixed-rate baseline only after enough sessions had accumulated, and change detection stayed a documented robust null when no candidate dominated the frontier. What the open-loop system never checked was whether the logged hours produced learning.

1.2 Problem Statement

The pace signal in Phase I comes from self-reported session logs. It records study time and says nothing about understanding. A learner can log every planned hour and still fail every test, while the plan keeps praising the pace and the examination result collapses. Existing tools leave the same gap open from the other side. Time-tracking applications record hours without judging understanding, and static plan generators never revise against any evidence at all. The missing piece

is assessment-based verification of genuine learning, wired into the planning loop so that grades drive the decision to stay the course or replan. Phase II closes this loop. It generates grounded assessments from the learner's own materials, grades them with an authoritative arm per question family, tracks mastery from the grade history, and feeds a verdict on the plan back into schedule regeneration.

1.3 Objectives

Phase II of the project pursues the following objectives.

1. Generate grounded assessments in three families. Each generation call produces one question in a single family, objective, written, or coding, grounded in retrieved chunks of the learner's material with recorded citations. A mixed run alternates the family per call, so every question stays traceable to the material it tests.

2. Grade each family with its authoritative arm. Objective questions grade deterministically against authored answers, written answers grade against a rubric applied by a language model, and coding answers execute in a sandboxed runner with hidden tests. The server grade is authoritative. Any client-side check stays advisory and never reaches the record.

3. Estimate mastery with Bayesian knowledge tracing and steer difficulty from it. A Bayesian model updates a per-skill mastery probability from the graded response history, and the practice system adapts the difficulty of upcoming problems to the estimate instead of serving a fixed sequence.

4. Guide with a Socratic ladder and a gated reveal. The coach answers questions through a ladder of Nudge, Hint, Targeted explanation, and Reveal, and the final step stays locked until the learner confirms. Guidance therefore never hands over the answer before the learner has worked.

5. Return advisory feedback copy on each graded attempt. A server endpoint drafts learner-facing feedback grounded in the attempt and its citations for updated entries, with a static fallback when the model is unreachable. The copy never alters the grade.

6. Run mixed practice through the library and practice hub. Assessments and practice runs live under the material library with cache-first loading, and a practice run resolves its questions lazily from the loaded assessments. Starting practice therefore stays fast even on slow networks.

7. Replan from the loop verdict and validate the loop on synthetic longitudinal histories. A loop endpoint reads mastery, posture, and grade evidence and returns a hold or replan verdict under a fixed confidence gate, and a verdict for replan carries into schedule regeneration with its identity attached. Four synthetic learner archetypes over five weeks each supply the longitudinal histories that a single-semester project cannot collect from real learners.

1.4 Scope of the Project

The work is delivered across two phases. **Phase I**, reported earlier, built and evaluated the open-loop system: pace calibration, change detection, completion projection, and capacity-based schedule regeneration, each compared offline against baselines and realised in the application. **Phase II**, reported here, closes the loop with assessment-based verification of genuine learning: grounded generation, authoritative grading, mastery tracing, Socratic guidance, advisory feedback, mixed practice runs, and verdict-driven replanning, validated on synthetic longitudinal histories. The target user in both phases is the individual self-directed learner. Multi-user, institutional, and instructor-mediated settings remain out of scope, as does deployment in a real classroom. The synthetic archetypes stand in for the longitudinal real-learner study a single semester cannot supply.

1.5 Organization of the Report

The remainder of the report is organized as follows. Chapter 2 surveys the research this phase draws on: Sections 2.1 to 2.4 cover knowledge tracing, item generation and tagging, grading, and guidance with feedback, Section 2.5 notes the sourcing method, and Section 2.6 summarizes the approaches and the gaps this project addresses. Chapter 3 specifies the system requirements in Sections 3.1 to 3.4. Chapter 4 describes the proposed methodology: the closed loop in Section 4.1, the selected models in Section 4.2, the comparison method in Section 4.3, the design evolution in Section 4.4, and the system architecture in Section 4.5. Chapter 5 gives the implementation details in Sections 5.1 to 5.5. Chapter 6 presents the results and discussion in Sections 6.1 to 6.5. Chapter 7 concludes in Section 7.1 and sets out future work in Section 7.2.

Chapter 2

Literature Survey

This chapter surveys the techniques this phase uses. Section 2.1 covers knowledge tracing and mastery models. Section 2.2 reviews automatic item generation and knowledge-component tagging. Section 2.3 describes grading methods for written and coded answers. Section 2.4 surveys Socratic guidance and learner feedback.

Section 2.5 notes the method used to source the surveyed work. Section 2.6 summarizes the approaches and states the gaps this project addresses.

2.1 Knowledge Tracing and Mastery

Tracing mastery from a short response history is the basis of the closed loop. Chen et al. [1] present a hierarchical Bayesian model of adaptive teaching and validate it through two online behavioral experiments with 312 adults. Hierarchical priors capture learner-to-learner variation without overfitting sparse data. This supports estimation of a single learner's state from few observations. The model targets a teacher's choice of examples rather than mastery of a tested skill. It supplies the prior structure but not the updater this project needs.

The updater is Bayesian Knowledge Tracing. It tracks a per-skill mastery probability through four fitted parameters: prior knowledge $P(L_0)$, learning rate T, guess G, and slip S. After a correct response the posterior follows

$$P_{\text{correct}}(L_n) = \frac{P(L_{n-1})(1-S)}{P(L_{n-1})(1-S) + (1-P(L_{n-1}))G}, \qquad P(L_n) = P_{\text{obs}}(L_n) + (1-P_{\text{obs}}(L_n))T. \quad (2.1)$$

The subscript names the observation just seen. The first term revises the estimate from the observed response. The second term moves the learner toward mastery by the fitted rate.

Zanellati et al. [2] review hybrid knowledge-tracing models in a taxonomy of approaches that combine Bayesian posteriors with machine learning. They conclude that hybrid models consistently outperform either pure Bayesian Knowledge Tracing or pure deep learning. Against this, the project's own bake-off pins its Bayesian variant, called bkt-v1, at an AUC of 0.888 against 0.889 for the baseline. That is parity rather than a win. The surveyed hybrids address correctness prediction on large logs. They do not steer the difficulty of what the learner sees next, which is the step this project adds with its adaptive chip.

Mai et al. [3] couple deep temporal modeling with a probabilistic layer in a Transformer-Bayesian hybrid that recovers interpretable, calibrated knowledge estimates. This keeps transparency without giving up predictive accuracy. The limitation for this project is cold start. A single self-directed learner arrives with almost no graded history. The calibrated pace work in Phase I already showed the pattern: a Bayesian estimate beats a fixed-rate baseline only after enough sessions accumulate.

The project therefore keeps the Bayesian updater with an informative stance from onboarding data. The mastery estimate then drives difficulty instead of a fixed sequence.

2.2 Automatic Item Generation and KC Tagging

The loop needs fresh questions on the learner's own material, each traceable to the passage it tests. The work on automatic item generation reports fluency first and traceability second. The large-language-model prompting study across STEM subjects [4] generates items in several subjects from prompts and checks them per subject. Against this, the project's own generation harness reports a groundedness of 0.867 with a near-duplicate rate of 0.004. The paper shows that generation covers subjects. It does not record which chunk each question came from, so a wrong question cannot be traced back to its source.

The self-hosted lecture-to-quiz study [5] generates multiple-choice questions with a local model and checks them with deterministic quality control. The control step rejects malformed items before they reach the learner. The project's retrieval baseline behind generation reports a recall at 3 of 0.7333 with a mean reciprocal rank of 0.7033 on the 754-chunk corpus. Deterministic checks catch shape defects. They cannot catch a well-formed question grounded in the wrong passage. Each generation call in this project therefore produces one question in a single family and records its chunk citations.

Tagging items to knowledge components faces the same split. The automated tagging study [6] generates knowledge-component tags from multiple-choice questions and attaches each item to the component it tests. The tag makes per-skill histories possible. The study stops at the tag. It does not use the tag to steer practice.

The project's mastery model needs exactly that signal, and its bake-off holds at the parity noted in Section 2.1. This project therefore carries a per-problem family label from generation into the run. Practice runs resolve their questions lazily from the loaded assessments instead of copying a fixed list.

2.3 LLM and Sandbox Grading

A grade that steers the plan must come from an authority the learner cannot sway. The EDUCON study on short-answer grading [7] applies a large language model with a rubric to answers in software engineering courses and grades them automatically. The rubric makes the judgment repeatable across answers.

Against this, the project's live matrix grades written runs 4 out of 4 with real rubric grades. A rubric grade alone cannot carry code. An answer there is right only if it runs. The project therefore keeps the model-judged arm for written answers and a separate execution arm for code.

Deterministic grading covers the objective family. The answer is authored at generation time and the grader compares it exactly, with no model call. The adjacent prediction-then-optimize study [8] predicts grades at an accuracy of about 0.73 before scheduling, which shows that grade-adjacent signals can drive a plan. Prediction is not grading. The project's live matrix takes objective runs 2 out of 2 and reviews 4 out of 4. The server grade stays authoritative, while any client-side check stays advisory and never reaches the record.

Code answers execute in a sandboxed runner with hidden tests. The runner is the self-hosted Piston service. The project grades coding runs 3 out of 3 live. The hidden pass rate decides the grade, and the reference solution never reaches the browser. The client runs visible tests only as advice.

The limitation this answers is single-signal grading treated as final. One arm cannot judge three families, so each family gets its own authoritative arm and hidden material stays server-side.

2.4 Socratic Guidance and Feedback

Guidance that gives the answer destroys the evidence the loop needs. The measurement side is well covered. Alhazbi et al. [9] review 25 empirical studies on measuring self-regulated learning from trace data and find that most validated indicators relate to time-management skills. This grounds schedule adherence and regularity as behavioral metrics. The review measures behavior. It does not guide the learner through a hard problem, which is where the Socratic ladder in this project starts.

Uzun et al. [10] show that a learner's engagement with analytics feedback is associated with both self-regulated-learning competence and course performance. That link motivates feedback that the learner actually reads. The project's guide runs 2 out of 2 live with real streamed steps. Feedback without a gate drifts into answer-giving. Each extra hint moves closer to the answer.

The project therefore answers through a ladder of Nudge, Hint, Targeted explanation, and Reveal. The last step stays locked until the learner confirms, so guidance never gives the answer before the learner works.

Help-seeking itself can be gamed. The think-aloud study of gaming the system [11] analyses self-regulated learning protocols and separates genuine attempts from moves that extract answers from the tutor. This matches the project's own replay result. After the fix the synthetic histories convert 4 out of 4 verdict-triggered replans with a material-scoped span of 32 days. Before the fix every case held.

The feedback copy in this project stays advisory for the same reason. A server endpoint drafts learner-facing text for updated entries with a static fallback, and the copy never alters the grade.

2.5 Methodology for Sourcing the Surveyed Work

The work above was assembled by advisor-guided curation with citation chaining, not by a fresh database sweep. The starting points were the Phase I knowledge-tracing trio and the implementation anchors of Phase II. These are single-family generation, per-family grading arms, the Socratic ladder, and advisory feedback. Each candidate was checked against four clusters matching Sec-

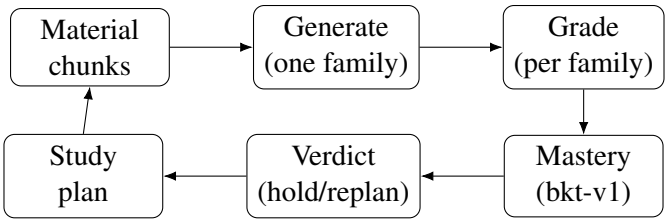


Figure 2.1: The assessment-verified closed loop this chapter motivates: generated items are graded per family, grades update mastery, the verdict steers the plan, and the plan points back at the material. Each stage is surveyed in Sections 2.1 to 2.4.

tions 2.1 to 2.4. Coverage across the four clusters mattered more than raw count. One representative work was kept per sub-theme, and redundant near-duplicates were dropped.

No new hundred-scale screening ran for this chapter, and none is claimed. Paper-side metrics appear where the source gives them verbatim, as in Sections 2.1 and 2.4. Where the Pillar-B sources state scope without a reusable number, the chapter states the scope. The project's own harness or live number appears beside it, labeled as the project's. This keeps every metric in the chapter traceable to the Phase I report or to live code reality, with no invented numbers.

2.6 Summary of Approaches

Table 2.1 summarizes the four families with their strengths and the limitations that motivate this work. Each technique is strong in isolation. None forms a closed loop from graded evidence back to the plan.

Table 2.1: Summary of Phase II approach families, with strengths and limitations for this project.

Approach family	**Strengths**	**Limitations for this project**
Knowledge tracing [1, 2, 3]	Per-skill mastery from short histories, interpretable, calibrated	Predicts correctness, does not steer difficulty
Item generation and KC tagging [4, 5, 6]	Fluent items at scale, deterministic checks, component tags	Untraceable to source chunks, tags unused for steering
Grading [7, 8] and sandbox execution	Rubric repeatability, deterministic match, hidden-test authority	Single-signal grades treated as final, client checks leak answers
Guidance and feedback [9, 10, 11]	Validated indicators, feedback the learner reads	Answer-giving tutors, grade-altering feedback

The gaps follow from the table. First, generation is fluent but untraceable, and tags exist without steering, so questions cannot yet drive per-skill practice. Second, grades come from one signal or from the client, so they cannot yet carry plan decisions. Third, guidance either measures without helping or helps by giving away the answer, so attempts stop being evidence.

This project answers all three with the loop in Figure 2.1. Generation is grounded and cited, and each family carries its own authoritative grade. Difficulty follows mastery, the ladder stays

gated, and feedback stays advisory. The verdict feeds schedule regeneration in Chapter 4.

Chapter 3

System Requirements Specification

This chapter specifies the requirements of the full system. Section 3.1 states the hardware environment. Section 3.2 states the software environment. Section 3.3 lists the functional requirements as numbered items, and Section 3.4 lists the non-functional requirements. The chapter assumes a single self-directed learner, a workstation with a graphics processor, and access to the hosted model APIs. It also assumes that synthetic longitudinal histories replace a classroom study, as stated in Section 1.4.

3.1 Hardware Requirements

The system runs on a single workstation. A single learner produces a small volume of data, and every statistical model trains and runs on a CPU. The embedding sidecar needs a graphics processor, so the workstation carries one. Table 3.1 lists the hardware environment. The graphics card named there is the project machine, not a minimum model.

Table 3.1: Hardware environment.

Component	Requirement
Processor	Any modern x86-64 CPU with a discrete GPU, project runs an AMD Radeon RX 9070 XT
Memory	8 GB RAM or higher
Storage	A few gigabytes for the container images and local data
Container runtime	Docker to run the intelligence service, the worker, the sidecar, and the sandbox
Client device	Any device with a modern web browser

3.2 Software Requirements

The system keeps the three Phase-I tiers and adds two containers. The browser client stays local-first. The Python intelligence service plus its worker now hosts generation, grading, mastery, guidance, and feedback. Supabase still provides authentication, the event database, and object storage. The graphics sidecar serves embeddings and rerank scores, and the sandbox runs learner code. Table 3.2 lists the software environment by tier.

Table 3.2: Software environment by tier.

Tier	Technologies
Client (single-page app)	React 19, Vite 5, TypeScript, Dexie over IndexedDB for local-first storage, date-fns
Intelligence service and worker	Python 3.12, FastAPI, Uvicorn, NumPy, PyJWT, generation, grading, mastery, guide, and feedback endpoints, OpenRouter generation provider plus Jev-1.13 judgment client, packaged with Docker
Backend services	Supabase: authentication, a PostgreSQL event database including `jev_review_flags`, and object storage
GPU inference sidecar	Qwen3-Embedding-0.6B plus Qwen3-Reranker-0.6B behind `/embed` and `/rerank`, separate compose project
Code-execution sandbox	Piston (`ghcr.io/engineer-man/piston`) on port 2000 for hidden-test grading
Offline research pipeline	Python (uv workspace), NumPy, with pytest and ruff for testing and linting
Tooling and deployment	pnpm workspace, Vitest and Playwright for testing, Vercel for hosting

3.3 Functional Requirements

The functional requirements state what the system does. Items FR-1 to FR-8 carry the Phase-I planner. Items FR-9 to FR-15 add the assessment-verified loop.

FR-1 The system shall let a learner define a study plan from a set of materials, a deadline, and a weekly study capacity.

FR-2 The system shall log study sessions, both from an in-application timer and from manual entry of a past session.

FR-3 The system shall estimate the learner's pace from the logged sessions using hierarchical Bayesian calibration.

FR-4 The system shall detect a shift in the learner's pace and surface a recalibration prompt for the learner to act on.

FR-5 The system shall project the completion date with a calibrated uncertainty band rather than a single date.

FR-6 The system shall regenerate the schedule from the learner's realized capacity when the learner chooses to replan.

FR-7 The system shall present the learner's progress, including the burn-up curve, streak, and schedule adherence.

FR-8 The system shall persist all data on the device and synchronize it to the cloud when a connection is available.

FR-9 The system shall generate one grounded question per call in a single family, with each question carrying its recorded chunk citations.

FR-10 The system shall grade each family with its authoritative arm. Deterministic marking applies to objective items, rubric grading to written items, and hidden-test pass rate to coding items.

FR-11 The system shall estimate mastery per skill with the `bkt-v1` model and recommend the next difficulty from that estimate.

FR-12 The system shall guide the learner through a Nudge, Hint, Targeted, and Reveal ladder and keep the full answer gated behind an explicit reveal.

FR-13 The system shall supply advisory feedback copy on updated results only, and that copy shall never alter a grade.

FR-14 The system shall run mixed practice sessions through the assessment library and the practice hub, with pause as leave-and-return.

FR-15 The system shall replan from the loop verdict once assessment evidence, not only logged hours, shows the plan is off track.

3.4 Non-Functional Requirements

The non-functional requirements state the qualities the system must exhibit. Items NFR-1 to NFR-6 carry the Phase-I qualities, with NFR-2 restated for the graphics processor. Items NFR-7 to NFR-9 add the measurable Phase-II qualities.

NFR-1 **Offline-first operation.** The core logging loop shall function without network connectivity, and analytical results shall be cached locally for reuse.

NFR-2 **Single-GPU scale.** The system shall serve the individual-learner workload on one workstation graphics processor, with no cluster and no multi-GPU setup.

NFR-3 **Per-learner isolation.** Each learner's data shall be isolated, using a per-user local database and row-level security in the cloud database.

NFR-4 **Authenticated services.** Requests to the intelligence service shall carry a signed token that is verified against the identity provider before any model runs.

NFR-5 **Responsiveness.** Calibration and projection results shall return within a few seconds, with a fallback to the last cached result when the service is unreachable.

NFR-6 **Behavioral alignment.** Where an algorithm exists in both the client and the service, the two implementations shall stay behaviorally aligned. Results shall not depend on which implementation runs.

NFR-7 **Grounded generation.** Every generated question shall carry its recorded chunk citations, and the harness measured groundedness at 0.867 on its gold set.

NFR-8 **Grade authoritativeness.** Hidden tests, reference solutions, and authored accepted values shall never reach the browser, and the server grade shall be final while client results stay advisory.

NFR-9 **Retrieval quality floor.** Retrieval shall hold recall at 1 of 0.67, recall at 3 of 0.90, and a mean reciprocal rank of 0.797 on the frozen 754-chunk corpus.

Chapter 4

Proposed Methodology

This chapter describes how the closed loop works and why it takes this shape. Section 4.1 presents the closed learning loop. Section 4.2 states the candidate models and the selected approach. Section 4.3 describes the comparison methodology. Section 4.4 traces the design evolution. Section 4.5 presents the system architecture.

4.1 The Closed Learning Loop

The loop runs over five stages. The learner plans through roadmap-engine bookings and studies through logged sessions. Practice and assessment produce server-graded evidence. Mastery and the loop verdict recalibrate the plan state. The verdict then drives the replan. Figure 2.1 shows the same loop at concept level.

The verdict is advisory and gated. `POST /v1/loop/decision` asks the hosted judge for one choice over the week's signals, the evidence summary, and the schedule posture. The code router keeps thin evidence, low confidence, and the middle band on `hold`. Only `suggest_replan` carries the verdict pair forward to the replan screen, and the committed `RoadmapReplanned` mirrors it. `Keep` writes no event.

Phase I ran this loop open. It recalibrated pace from logged sessions and raised a prompt, and the learner decided whether to replan. Nothing checked that study produced understanding. Phase II verifies first: grades steer mastery through Equation 2.1, mastery and posture steer the verdict, and the verdict steers the plan.

4.2 Candidate Models and Selected Approach

The project fixed each stage by comparing candidates against evidence instead of by default. This section states the candidates and the selected approach per stage. Section 4.3 describes how the comparison ran. Table 4.1 summarises the selections.

4.2.1 Item Generation

The generator had to produce questions the project can trace. The project rejected multi-family single-call generation without recorded provenance. The selected shape is one family and one question per call, grounded in retrieved chunks with recorded citations. The server gate enforces it: `formats` must be one of `objective`, `written`, or `coding`, and `questionCount` must be `1`. A mixed run alternates families across calls instead of mixing them in one call.

4.2.2 Grading

One signal cannot judge three families, so each family carries its own authoritative arm. Objective answers are authored at generation time and compared exactly. Written answers go to a rubric-judged model arm. Code answers execute in the Piston sandbox against hidden tests, and the reference solution never reaches the browser. Every client-side check stays advisory and never enters the grade record.

4.2.3 Mastery and Difficulty

The mastery candidates were the deep and hybrid variants surveyed in Section 2.1. The bake-off held the Bayesian variant at parity with the baseline, so the project kept it and named it bkt-v1. The estimate updates per skill through Equation 2.1. The adaptive chip then maps the estimate to a difficulty recommendation.

4.2.4 Guidance and Feedback

Guidance that gives the answer destroys the evidence the loop reads. The project rejected an open tutor that reveals on demand. The selected shape is the ladder of Nudge, Hint, Targeted explanation, and Reveal, with the last step locked until the learner confirms. Feedback copy stays advisory in the same way: a server endpoint drafts learner-facing text for updated entries through the `feedback-v2` template with a static fallback, and the copy never alters the grade.

4.2.5 Loop Verdict

The project weighed a raw model choice against a gated router over it. The project kept the gated pair: the Jev judge (`jev-1.13` through a hosted API) proposes, and the code router disposes. Thin evidence, low confidence, and the middle band all land on `hold`. The gate constant `LOOP_APPROVE_AT` is `0.7`. Only a confident call with sufficient evidence passes it.

Two honest limits stay on record. The coding arm refuses material it cannot ground a question in, and reports `code_not_derivable` instead of inventing a question. The Phase I estimators carry over unchanged: the single-arm CUSUM still prompts recalibration, and the split-conformal interval correction stays a validated refinement that is not shipped.

Table 4.1: Candidate approaches per loop stage and the approach selected for the delivered system.

Stage	Candidates compared	Selected (delivered)
Item generation	Multi-family single call, ungrounded prompting, single-family grounded	Single family and single question per call, chunk citations recorded
Grading	Advisory client-side check, single-signal grade	Server-authoritative arm per family; hidden tests never reach the browser
Mastery and difficulty	Deep variants, hybrid models, fixed sequence	bkt-v1 with the adaptive difficulty chip
Guidance and feedback	Open tutor, grade-altering feedback	Gated ladder with locked reveal; advisory-only `feedback-v1` copy
Loop verdict	Raw model choice	Jev `jev-1.13` with the fail-closed router (`LOOP_APPROVE_AT` 0.7)

4.3 Comparison Methodology

The comparison rests on two legs. The offline leg scores components against fixed golds. The longitudinal leg scores the whole loop on synthetic histories with known posture. Both legs ran before selection, and the live browser matrix only checks deployment afterward.

The offline leg is the generation-quality harness. It holds golds in `eval_golds.json` with three fold hashes at $K = 5$, and its gate stays fail-closed by default. It reports groundedness at 0.867 with a near-duplicate rate of 0.004. Retrieval recall at 3 is 0.733. The bkt-v1 AUC is 0.888 against 0.889 for the baseline. Coding derivability reads 0.0 on the golds.

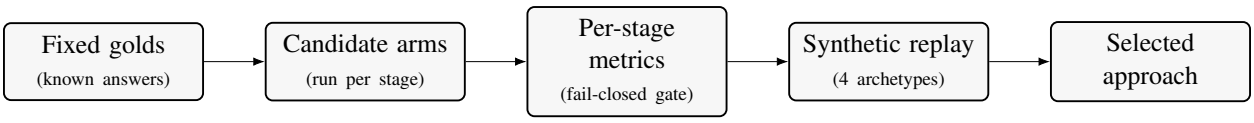


Figure 4.1: The two-leg comparison. Fixed golds score each candidate arm behind a fail-closed gate; the synthetic replay then scores the whole loop before the approach is selected.

The longitudinal leg replays four synthetic archetypes: steady, crammer, struggling, and returning. Development histories stay separate from evaluation histories, so a win reflects behaviour instead of fitting. Each run ends in a loop verdict joined to its replan, and the project scores conversion with next-week adherence. The post-fix replay converts 4 out of 4 `suggest_replan` verdicts into verdict-linked replans over a material-scoped span of 32 days. Confidence reads 1.0 with sufficiency from 0.66 to 0.77. Every replan extends the deadline.

The live matrix confirms deployment. It never promotes a candidate. Objective runs take 2 out of 2, written runs grade 4 out of 4 with real rubric grades, and coding runs grade 3 out of 3.

Table 4.2: Evaluation metrics by loop stage.

Stage	Metrics
Item generation	Groundedness in the cited chunks, near-duplicate rate
Grading	Live pass counts per family (objective 2 of 2, written 4 of 4, coding 3 of 3)
Mastery and difficulty	Area under the curve against the baseline
Guidance and feedback	Live pass counts (guide 2 of 2, feedback loop 3 of 3); copy never alters the grade
Loop verdict	Verdict-to-replan conversion (4 of 4 post-fix), next-week adherence rebound

4.4 System Design and Evolution

The system reached this shape in three moves. The starting point is the delivered Phase I design: capacity-based bookings with an open loop, where the closed-loop feedback from verified mastery stayed a dashed extension. The base scheduling study [8] still supplies the predict-then-optimize contrast that design answered.

The second move inserted assessment without touching the booking model. Generation, grading, mastery, guidance, and feedback landed in the intelligence service beside the Phase I calibration routes. Bookings, session logging, and the local-first store did not change. The third move closed the loop: the verdict gate, the verdict pair mirrored onto `RoadmapReplanned`, and the synthetic replay proving `suggest_replan` converts to a replan.

This section states the path once. The problem it answers sits in Section 1.2, and the capability list it delivers sits in Sections 3.3 and 3.4. Nothing here restates either.

4.5 System Architecture

The deployed system keeps the Phase I split. Computation that fits a model to history runs in the Python service. Computation that derives a view from the local event log runs in the client. Supabase sits between them as the data hub with row-level isolation per learner. Figure 4.2 shows the tiers with the Phase II additions solid.

The browser client owns bookings, projection, mastery readers, and the verdict handoff. The intelligence service with its worker owns generation, grading, mastery, guidance, feedback, and the loop verdict. The Jev judge sits folded inside as a hosted-API client instead of a separate tier. The graphics sidecar serves embeddings and rerank scores from its own compose project, and the sandbox runs learner code on demand. Table 3.2 lists the full tier inventory.

Figure 2.1 states the concept the literature motivates, and this figure states where each stage runs and which wire joins them. The loop that stayed dashed in Phase I runs solid here.

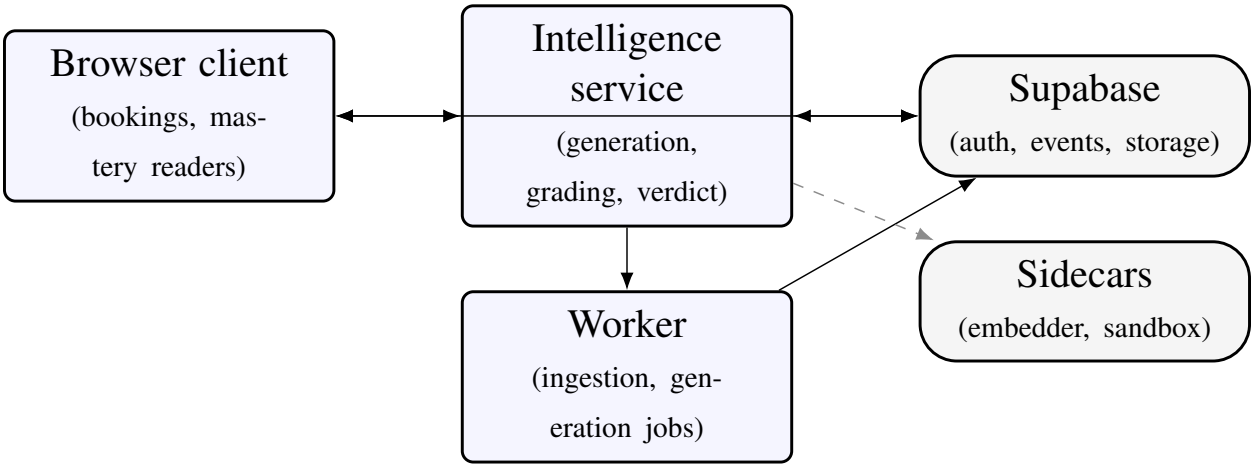


Figure 4.2: The deployment that realises Figure 2.1. The client derives views from the local log, the service fits models to history with the judge folded inside, Supabase hubs the data, and the sidecars serve on demand.

Chapter 5

Implementation Details

This chapter describes how the Phase-II system is built. Section 5.1 records the development environment. Section 5.2 describes the datasets and their sources. Section 5.3 describes the model and service implementation. Section 5.4 states the metrics and the steps taken for reproducibility. Section 5.5 presents the delivered Phase-2 application.

5.1 Development Environment

The project keeps the Phase-I repository shape. A pnpm workspace holds the TypeScript client with its shared engines. A uv workspace holds the Python packages, the service, and the research code. The two coexist so that the research code imports the same Python engines the service runs. That keeps the evaluated code and the shipped code identical. Table 5.1 lists the environment. The first eleven rows carry the Phase-I stack unchanged. The last six rows add the Phase-II generation, judgment, grading, embedding, and editor dependencies.

5.2 Datasets and Their Sources

A single learner's real log cannot supply a known ground truth. The Chapter 6 comparison therefore runs on fixed artefacts with planted and known properties. Table 5.2 lists the datasets and the role each plays.

The frozen corpus is a single textbook at 754 chunks with 768-dimensional sidecar vectors. Three gold files score the offline leg against it: the objective probe set, the written gold set, and the coding gold set. The fixed folds and the attempt gate sit beside them in `eval_golds.json`. The longitudinal leg replays four synthetic archetypes with known posture: steady, crammer, struggling, and returning. Their generator plants weekly pace, skip behaviour, gap weeks, and replan back-pointers explicitly. The Open University dataset calibrates the Phase-I generator only and does not enter the Phase-II comparison. Real single-learner logs stay reserved for future validation and are not used here.

Table 5.1: Development environment: tools and libraries. The first eleven rows carry the Phase-I stack; the last six add the Phase-II loop.

Concern	Technology
Client language and build	TypeScript, Vite, pnpm workspace
Client UI	React 19
Local-first storage	Dexie over IndexedDB
Intelligence service	Python 3.12, FastAPI, Uvicorn
Scientific stack	NumPy, SciPy
Service authentication	PyJWT, verifying Supabase-issued tokens
Containerisation	Docker via Colima
Python workspace	uv
JavaScript testing	Vitest, fast-check (property tests), Playwright (end-to-end)
Python testing and linting	pytest, ruff
Backend and hosting	Supabase (auth, database, storage), Vercel
Generation provider	OpenRouter; `deepseek/deepseek-v4-flash-0731`, temperature 0.3, reasoning off, 30 s timeout
Feedback-copy model	`meta/muse-spark-1.3-contributor` through `FEEDBACK_MODEL`, reasoning effort low
Judgment client	Jev-1.13 hosted System One API through `OPENROUTER_JEV_API_KEY`; in-process client
Code-execution sandbox	Piston sandbox profile on port 2000 for hidden-test grading
Embedding sidecar	Qwen3-Embedding-0.6B and Qwen3-Reranker-0.6B on port 8200; separate compose project
Client code editor	CodeMirror with an advisory Pyodide check that never grades

Table 5.2: Datasets used in Phase II and their sources.

Dataset	Source	Role
Frozen corpus and harness golds	In-repository gold files: `eval_golds.json` with probe, written, and coding sets; ACCA text at 754 chunks with 768-dimensional sidecar vectors	Primary ground truth for the offline leg: generation, retrieval, mastery, and coding checks
Synthetic longitudinal histories	In-repository generator with four archetypes: steady, crammer, struggling, and returning	Whole-loop replay with known posture; post-fix replay evidence versioned
OULAD and real logs	Public Open University dataset and the candidate's own sessions	Calibration source for the Phase-I generator and reserved validation; neither enters the Phase-II comparison

5.3 Model and Service Implementation

The algorithms stay implemented once in behaviour and twice in code where the client must render them. The client-side engines are written in TypeScript, and a Python mirror of each engine runs behind the intelligence service. A golden-fixture parity harness keeps the two implementations aligned. The TypeScript engines emit reference inputs and outputs. A Python test replays each input and asserts numeric equality to a tight tolerance. The shared numeric constants, such as the BKT parameters, stay identical on both sides.

The split between client and service follows the architecture of Figure 4.2. Service-side code owns grounded generation, per-family grading arms, and the mastery endpoint. It also owns the guide stream with its gated reveal. It further owns the feedback-copy endpoint and the loop verdict with the Jev judge folded inside as a hosted-API client. The client keeps the engines it must read locally. Those are the roadmap bookings, the Gaussian-process projection, the mastery readers with the difficulty chip, and the verdict handoff. The offline research harness imports the production Python engines as candidates. It never re-implements them, so a result measured offline is a result about the shipped code. Listing 5.1 shows the shipped BKT forward filter, which realises Equation 2.1, and Table 5.3 lists the main packages.

Listing 5.1: The shipped BKT forward filter, which realises Equation (2.1).

```
def bkt_forward(
    observations: list[MasteryObservation],
    params: BktParams = DEFAULT_PARAMS,
) -> tuple[list[float], list[float]]:
    """Run the forward filter over one skill's observation sequence.

    Returns ``(mastery_after, p_correct_before)`` parallel lists:
    ``p_correct_before[i]`` is the predicted probability of correctness for
    observation ``i`` computed from the belief before it, and
    ``mastery_after[i]`` is P(mastery) after processing observation ``i``.
    """

    mastery: list[float] = []
    p_correct: list[float] = []
    belief = params.p_init
    for obs in observations:
        p_correct.append((1.0 - params.p_slip) * belief + params.p_guess * (1.0 - belief)
    )
        if obs.correct:
            numerator = (1.0 - params.p_slip) * belief
            denominator = numerator + params.p_guess * (1.0 - belief)
        else:
            numerator = params.p_slip * belief
            denominator = numerator + (1.0 - params.p_guess) * (1.0 - belief)
        belief = numerator / denominator if denominator > 0 else belief
        belief += (1.0 - belief) * params.p_learn
        mastery.append(float(np.clip(belief, 0.0, 1.0)))
    return mastery, p_correct
```

Table 5.3: Main packages and their roles.

Package / module	Role
`packages/progress` (TS)	Client mastery engine with bkt-v1: projection and the difficulty chip
`packages/roadmap-engine` (TS)	Client booking and roadmap generation; the booking model is unchanged
`packages/py-progress` (Py)	Python mirror: Bayesian and enriched-shrinkage calibration, CUSUM, Gaussian process, bkt-v1
`packages/py-roadmap-engine` (Py)	Python mirror of the roadmap engine
`services/intelligence` (Py)	FastAPI service with a portless worker: generation, grading, mastery, guide, feedback, and the loop verdict
`services/embedder`	Qwen3 embedding and rerank sidecar in a separate compose project
Piston sandbox	External image for hidden-test code execution, started on demand
Research scripts	Offline harness and probes importing the production engines as candidates

5.4 Evaluation Metrics and Reproducibility

The metrics used to score each loop stage are defined in Table 4.2 of Chapter 4. This section records how the project keeps them reproducible instead of redefining them.

Three facts carry the reproducibility claim. First, every gate fails closed by default. The evaluation harness stays inert unless the caller passes `-activate`. The loop router maps a judge failure onto `hold` through a typed service error. The grading arms return a deterministic fail-closed outcome for unusable input and never fabricate a score. Second, every call carries one request identifier from the browser to the service logs. The client sends `X-Request-ID` on each intelligence call. The service echoes it on each response, and every error log line carries it. One identifier therefore joins the frontend, the backend, and the end-to-end evidence. Third, a reader can re-run both legs from versioned files. The offline leg needs `eval_golds.json` with `eval_harness.py`. The longitudinal leg needs the synthetic generator with the four archetypes and the versioned post-fix replay evidence.

5.5 The Phase-2 Application

The working application realises the selected models. This section presents that application in full. Phase-I delivered the planner with its session runtime, progress views, and roadmap lifecycle, and that substrate is unchanged. This section follows the five Phase-II capabilities in order. The requirements behind them sit in Sections 3.3 and 3.4, and nothing here restates them. A flow diagram builds up across the subsections, adding one stage at a time.

5.5.1 Materials and Ingestion

A material enters through the create page. The source grid offers plain text, PDF upload, and web article side by side. The pipeline turns the upload into embedded chunks. The library card reports each state honestly until the material reads ready. A ready card offers practice instead of a status badge. A failed card keeps its badge with a retry action. The detail page owns the rest of the lifecycle. It shows provenance and the outline, and it links onward to assessment and practice. Figure 5.1 shows the flow at this first stage.

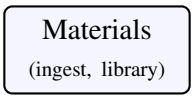


Figure 5.1: Application flow after Section 5.5.1: materials reach readiness in the library.

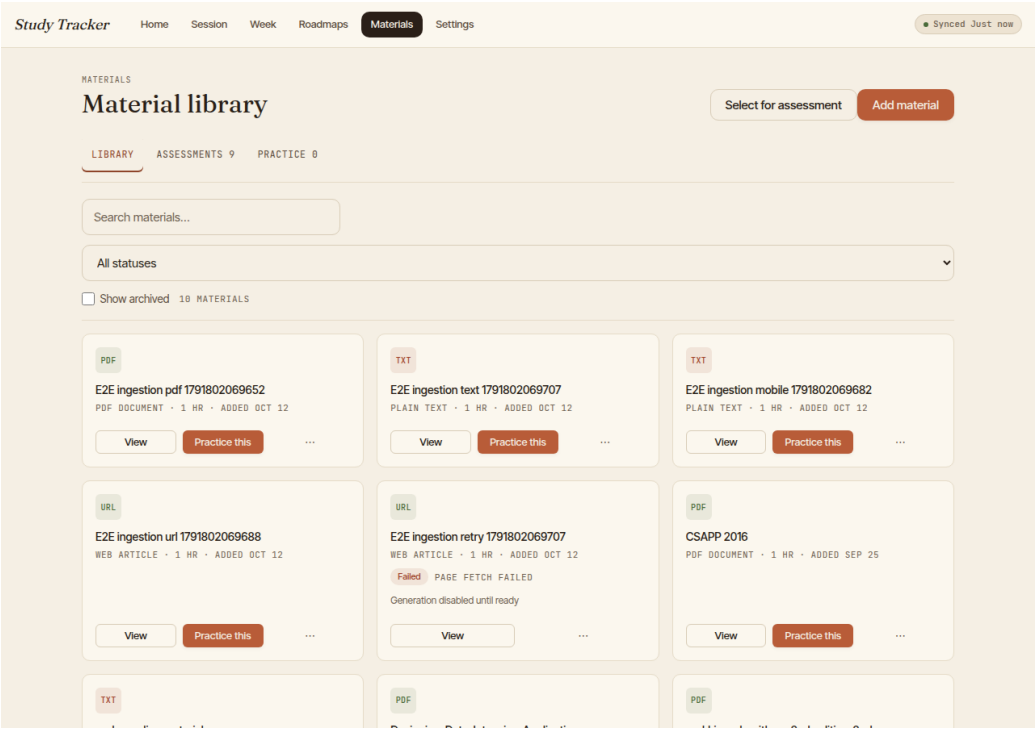


Figure 5.2: The material library: ready cards offer practice while the failed card keeps its badge with a retry action.

5.5.2 Assessment Generation

A generation run starts from the config page. The page binds one material, one family, and one scoped chapter before it requests a single question. The server gate admits exactly one family and one question per call. A mixed run alternates families across calls. The assessment library lists every attempt with its family and status. Cache-first rows paint before revalidation completes. Generation runs as a tracked job. Only a run still marked generating retries in place, and a failed run ends there. Generation extends the flow in Figure 5.3.

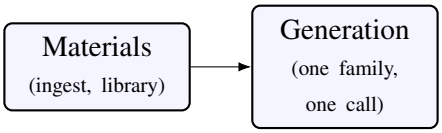


Figure 5.3: Application flow after Section 5.5.2: generation binds one material, one family, and one chapter per question.

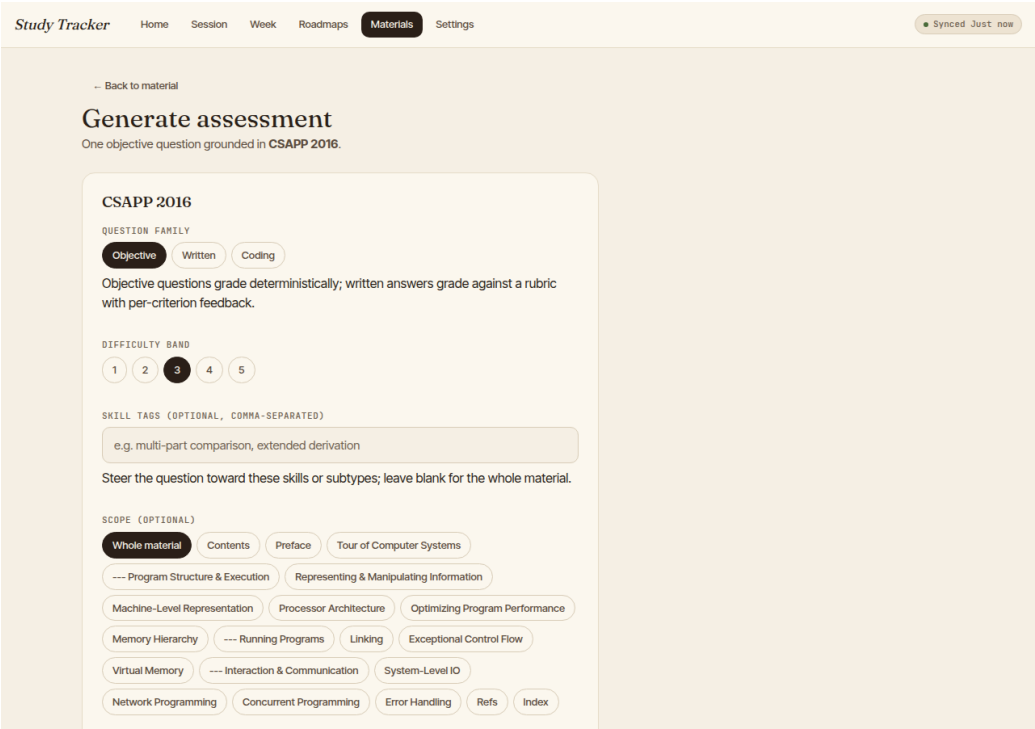


Figure 5.4: Assessment generation: one material, one family, and one scoped chapter per question.

5.5.3 Taking and Review

The taking screens collect one answer per question. They never touch hidden tests, reference solutions, or authored accepted values. Objective answers compare exactly against values authored at generation time. Written answers go to the rubric arm, and coding answers execute against hidden tests. Coding runs in an embedded editor with an advisory client check. The server grade stays final, and no client result enters the record. The review module shows each attempt beside its grade with per-question detail. Figure 5.5 then covers taking and review.

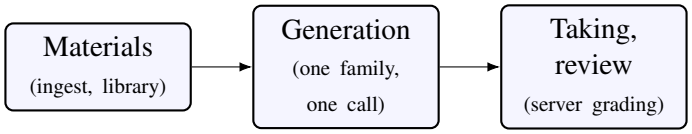


Figure 5.5: Application flow after Section 5.5.3: taking collects answers and review shows the server grade.

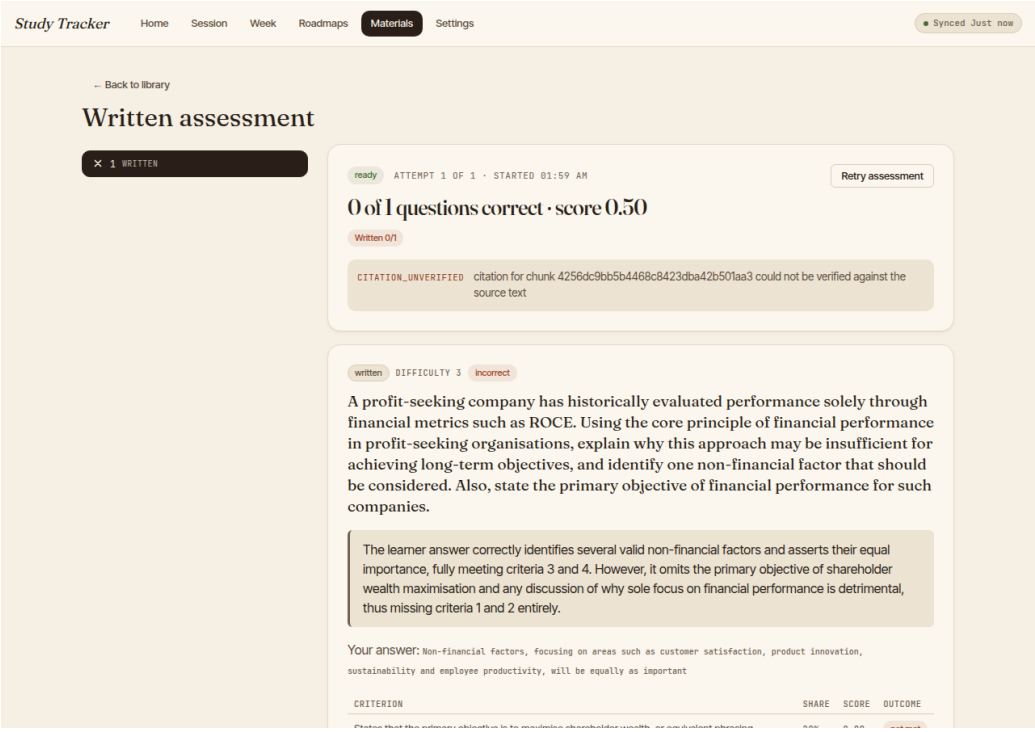


Figure 5.6: Taking and review: the server grade with per-criterion rubric feedback.

5.5.4 Practice and Guidance

The practice hub gathers runs across materials. Each run resolves its questions lazily from the loaded assessments and stores no copies. Pause means leave and return. Neither finish nor abandon invents an observation. The Socratic coach stays anchored to the page. It streams Nudge, Hint, and Targeted guidance against retrieved chunks and locks Reveal behind an explicit confirmation. The

mastery reader beside it shows the current estimate with its difficulty recommendation. Practice and guidance join the flow in Figure 5.7.

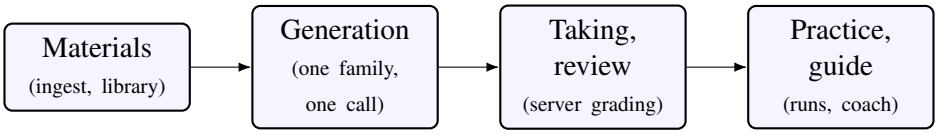


Figure 5.7: Application flow after Section 5.5.4: practice runs resolve lazily and the coach guides per attempt.

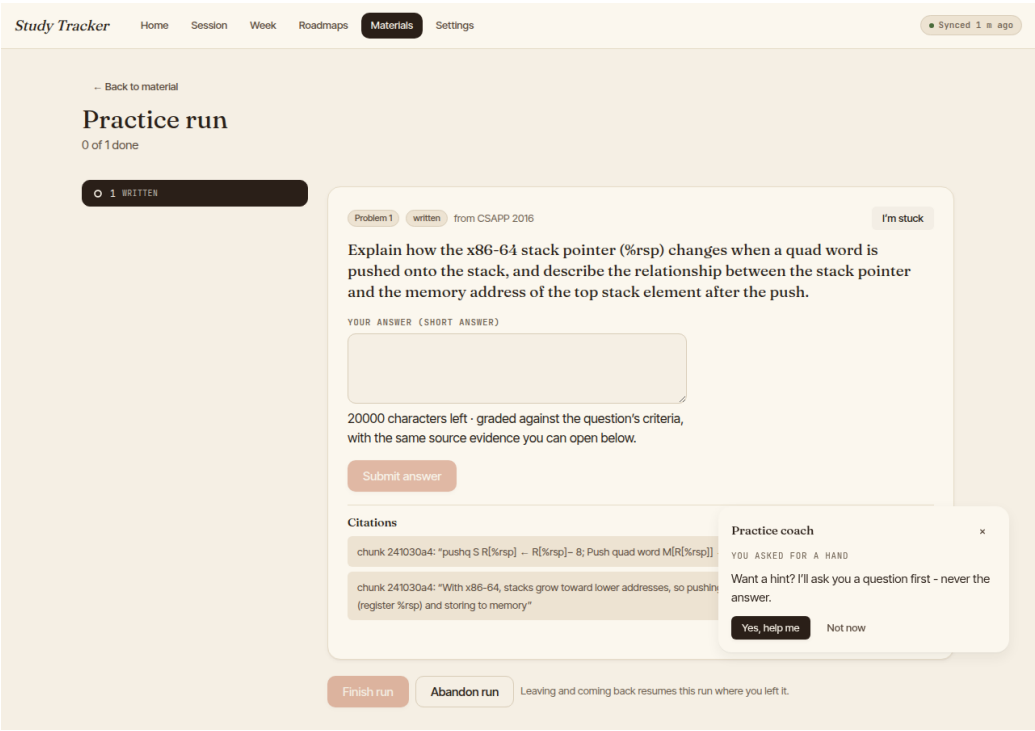


Figure 5.8: Practice run: the lazily resolved question with the Socratic coach anchored to the attempt.

5.5.5 Feedback and the Closed Loop

The roadmap detail page carries an evidence ledger. The ledger reads cold, updated, stale, or rebuilding from the cached mastery and the latest copy. The feedback-copy endpoint drafts text for updated entries only. The static fallback covers every other state, and the copy never alters a grade. The loop verdict proposes one choice over the week. Only `suggest_replan` hands its verdict pair to the replan screen. Confirming a replan mirrors the pair onto `RoadmapReplanned` and keeps the plan identity. `Keep` writes nothing. Figure 5.9 completes the chain with feedback and the loop verdict.

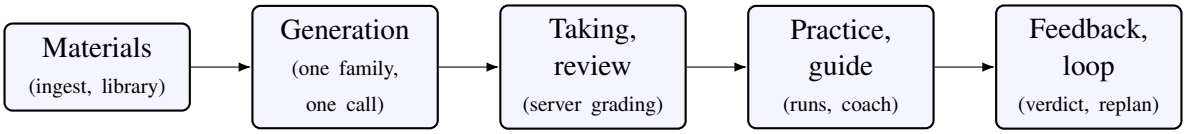


Figure 5.9: Application flow after Section 5.5.5: the verdict closes the loop onto the replan.

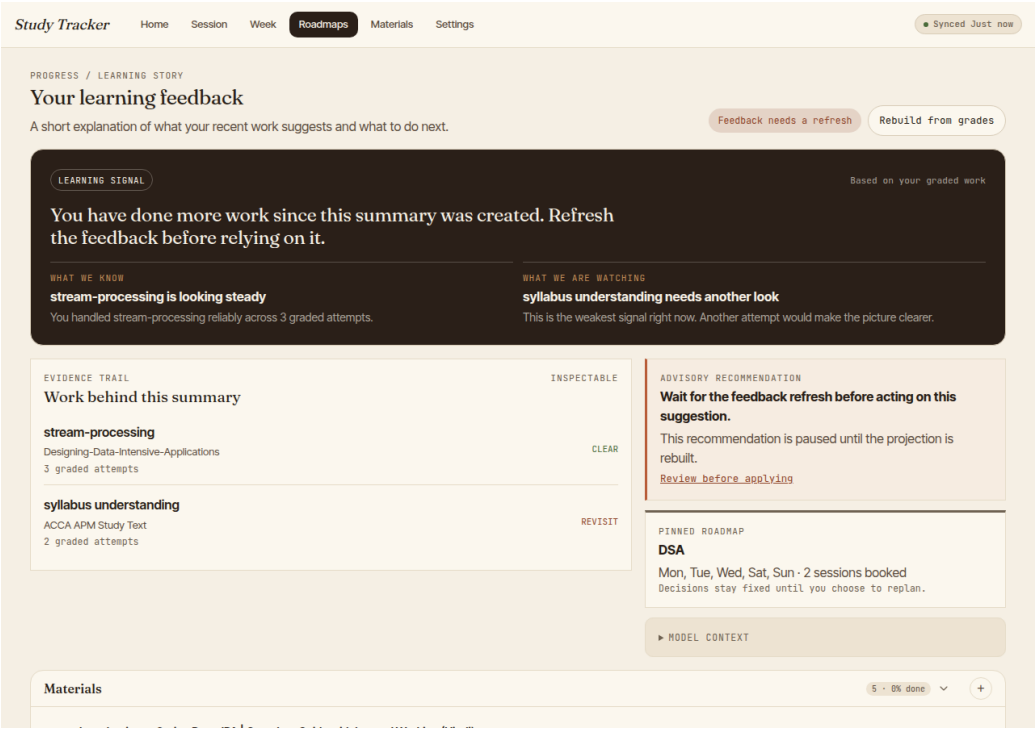


Figure 5.10: Feedback and the loop: the inspectable evidence trail with the advisory recommendation.

Chapter 6

Results and Discussion

This chapter reports the offline comparison for each Phase-II loop stage and discusses what the numbers mean for the delivered system. All offline numbers run on the fixed artefacts described in Section 5.2, under the comparison design of Section 4.3. The metrics are those of Table 4.2; this chapter reports against them without redefining them. Section 6.1 reports generation quality with the retrieval, generation, and tracing tables. Section 6.2 reports live verification as a deployment check. Section 6.3 reports the closed-loop verdict. Section 6.4 reports synthetic longitudinal validation. Section 6.5 discusses the results together.

6.1 Generation Quality

The offline leg scores components against fixed golds behind the fail-closed gate of Figure 4.1. Three tables carry the leg: retrieval with ranking, generation with its grading signals, and tracing with calibration. Reproducibility for both legs rests on Section 5.4. The harness stays inert unless activated, every call carries one request identifier, and both legs re-run from versioned files.

Table 6.1 reports recall and MRR on the coherent 754-chunk ACCA split, the live endpoint row, and the DDIA second corpus. The harness dense and hybrid arms reproduce the recorded in-process baselines exactly, which validates the rerank arm beside them. Reranking lifts hybrid recall at 1 by 7 points (0.60 to 0.67) and recall at 3 by 10 points (0.80 to 0.90). MRR rises by 0.077 (0.720 to 0.797). The live endpoint reproduces the plain RPC arm exactly (0.60/0.73/0.87/0.703) and its reranked row reads 0.70/0.90/0.93/0.813. It is to be noted that the endpoint reranked row sits above the harness arm because the RPC candidate set differs from the in-process copy. The DDIA corpus moves the same direction with larger magnitude. Reranking lifts recall at 1 by 16 points (0.47 to 0.63) and recall at 3 by 17 points (0.73 to 0.90). MRR rises by 0.139 (0.660 to 0.799). Cross-corpus absolutes are not comparable because the two gold sets were authored independently; only the arm deltas transfer. The retired 788-split headline (recall at 1 of 0.77, recall at 3 of 0.97, MRR of 0.858) is the frozen-split precedent only. Two rejected arms stay on record: title-prefix indexing shows no gain, and int8 quantisation collapses to 0.10 across recalls at 2.45 times speedup.

Generation runs on the S-layer aggregate (n=180); Table 6.2 carries the judge and the coding pair. Groundedness reads 0.8667 (26 of 30 judged answerable) against a gold support of 0.7803. The judge is `deepseek/deepseek-v4-flash-0731` with reasoning off. The two numbers measure different things. The judge scores whether the cited chunk supports the answer, while gold

Table 6.1: Retrieval: recall and MRR by arm. ACCA panels share the 30-question harness; the endpoint row is the live deployed path; DDIA is the second corpus.

Corpus and arm	r@1	r@3	r@5	MRR
ACCA 754-chunk harness				
dense	0.60	0.77	0.87	0.706
hybrid	0.60	0.80	0.87	0.720
hybrid_rerank	0.67	0.90	0.93	0.797
Live endpoint, same corpus				
reranked	0.70	0.90	0.93	0.813
DDIA 1094-chunk second corpus				
dense	0.40	0.73	0.83	0.593
hybrid	0.47	0.73	0.83	0.660
hybrid_rerank	0.63	0.90	0.97	0.799

support scores whether the citation contains the gold string. Schema validity reads 0.9611 and citation validity 0.9133, with a near-duplicate rate of 0.0044 at a cost of $0.061289. The coding arm refuses material it cannot ground, so derivability on the golds reads 0.0 as the honest terminal. The fresh live pull derives at 0.5349 over 43 judged attempts and passes the 0.5 gate. Gold agreement reads 3 of 3. The gold counts are 30 objective, 6 written, and 4 coding (3 derivable plus 1 not), with attempt gate K=5 over 3 fold hashes.

Table 6.2: Generation: S-layer aggregate, groundedness judge, and coding pair. Judge and citation metrics are distinct by construction.

Metric	Value
Schema valid (n=180)	0.9611
Citation valid	0.9133
Gold support (citation-contains-gold)	0.7803
Near-duplicate rate	0.0044
Cost (USD)	0.061289
Groundedness, LLM-judged (26 of 30)	0.8667
Coding derivable on golds (honest terminal)	0.0
Coding derivable live (43 judged, gate 0.5)	0.5349

Tracing accuracy across three populations sits in Table 6.3. The bake-off fixed the shipped parameters (p_init 0.15, p_learn 0.1, p_slip 0.05, p_guess 0.2) at mean AUC 0.785 with ECE 0.057. The live harness over 282 observations across 139 skill sequences gives bkt-v1 AUC 0.7397 with ECE 0.1401 against running-proportion 0.7368 with 0.2679. The external pyBKT accoding anchor reads 0.602 to 0.611 under gate `masteryMinAuc` 0.6. The reading is parity, not a win. Bkt-v1 matches its baseline on ranking while the calibration error nearly halves on live attempts. The Chapter 4 bake-off pair (0.888 against 0.889) says the same on its cut. Difficulty calibration is recorded as unmeasured. The current pool holds 23 observations on a single question in one band, so the band error is null by construction. The adaptive chip stays uncalibrated-advisory. The update itself is Equation 2.1 with the shipped filter in Listing 5.1.

Table 6.3: Knowledge tracing: AUC and calibration across three populations. Framed as parity, not a win.

Population	AUC	ECE / note
Synthetic bake-off (shipped params)	0.785	0.057
Live harness bkt-v1 (282 obs, 139 seqs)	0.7397	0.1401
Live harness running-proportion	0.7368	0.2679
External pyBKT accoding anchor	0.602–0.611	gate 0.6

6.2 Live Verification

The live matrix confirms deployment and never promotes a candidate, per Section 4.3. Objective runs take 2 of 2, written runs grade 4 of 4 with real rubric grades, and coding runs grade 3 of 3. Guide runs pass 2 of 2, practice runs pass 2 of 2, and feedback copy passes 3 of 3. Mastery passes 2 of 2 and review passes 4 of 4. Ingestion passes 5 with 2 skipped. Every run reports zero console and page errors. AC1, AC3, and AC4 pass with AC2 passing with skips. Each call joins frontend to service logs on one request identifier per Section 5.4.

At scale, the full-book E2E runs through the sidecar. The 572-page fixture (about 23 MB) moves from upload to created at 13.5 s and extracting at 15 s. Embedding reads 48 s and ready reads 101 s. The start-to-ready wall is about 88 s at 10 s poll granularity. The same fixture yields the durable 754-chunk baseline with 0 NULL embeddings. The material was deleted after the run with the card-gone state verified, so the shared account sits as found.

6.3 Closed-Loop Decision

Table 6.4 is the paper's verdict evidence: the post-fix replay converts 4 of 4 `suggest_replan` verdicts into verdict-linked replans. Confidence reads 1.0 with sufficiency from 0.66 to 0.77 across the four archetypes. Every replan extends the deadline and mirrors the exact verdict pair onto `RoadmapReplanned`. The span reads 32 days for all four from the durable material-scoped `SessionLogged.payload.date`. The pre-fix contrast is 4 of 4 `hold` at confidence 0.18 to 0.34 on spanless neutral inputs. The gate moved from 0.9 to 0.7. The sweep over 21 cases agreed at 0.857 across 0.5 to 0.7. The 0.8 setting was falsified by the live 0.79 dip. The JEV cutoff and margin prior of 0.6 and 0.10 holds. The 0.7 training favourite contradicts the test pair, and the test met-class stays thin at 2 pairs. Hook checks pass 37 of 37 and feedback checks pass 77 of 77 with typecheck and lint clean.

Table 6.4: Loop verdict: post-fix replay across four archetypes. Per-user sufficiency and decidedAt values sit in the versioned JSON beside the replay evidence.

Archetype	Verdict	Conf	Suff	Option	Converted
steady	suggest_replan	1.0	0.66–0.77	extend-deadline	yes
crammer	suggest_replan	1.0	0.66–0.77	extend-deadline	yes
struggling	suggest_replan	1.0	0.66–0.77	extend-deadline	yes
returning	suggest_replan	1.0	0.66–0.77	extend-deadline	yes

6.4 Synthetic Longitudinal Validation

The longitudinal leg replays four synthetic archetypes with known posture: steady, crammer, struggling, and returning. Development histories stay separate from evaluation histories, so a conversion reflects behaviour instead of fitting. The full 24-row adherence emission and the per-user decidedAt values sit in Appendix A; this section reports the headline with its caveat.

Next-week adherence measures 0.000 for all four in the post-replan extension week. Capture ran one day before the deadline. No sessions were logged after the replan, so the zero is a measured zero with that rebound-era caveat, not a learning result. The 12 mastery rows read after at about 0.1067 throughout, below the prior of 0.15 by construction. The seeded scripts answer about 90 percent wrong because they prioritise grading-path coverage over learner success. Those rows measure the scripts and are never cited as tracing validation; the tracing evidence lives in Table 6.3. Each of the 4 verdicts joins its `RoadmapReplanned` by the exact verdict pair with createdAt at or after decidedAt. A seed-shape finding is recorded with the evidence. The seed writes one timestamp with a zone offset while the booking reference uses milliseconds. The strict-equality join silently mistraces until emission canonicalises the instant. The product is untouched.

The dip-window replay pins the browser clock to the week-3 trough for every archetype. All four hold at confidence 0.28 to 0.35 with sufficiency 0.66 to 0.77 and convert nothing, because `Keep` writes nothing. A September-12 pin for the struggling archetype flips the choice to `suggest_replan` at 0.61 to 0.69 but the gate holds it. The gradient reads as intended: trough holds on the merits, week 4 proposes below the gate, and the rebound eve converts at 1.0. The replay drove verdicts only and produced no new graded attempts, so the difficulty pool is unchanged. Cloud counts sit unchanged after all runs with no new writes and no synced markers. Four caveats travel with this leg. The clock pins to 2026-10-14, and capture covers the rebound era rather than the week-3 dip. The onboarding gate is local only, and cold restore duplicates locally while extracting from cloud only.

6.5 Discussion

Read together, the results separate into honest nulls, real wins, and validated refinements not yet shipped. Tracing is a null on ranking and a real improvement on calibration. AUC holds at parity while the calibration error nearly halves on live attempts. Coding on the golds is an honest terminal, not a failure. The arm refuses rather than invents, and the live pull passes its gate. Difficulty calibration and next-week adherence are unmeasured and measured-zero respectively, each with its named cause rather than a softened adjective. The clear wins are four. Retrieval ranking rises on both corpora in the same direction. Grounded generation reads 0.8667 with near-duplicates at 0.0044. The verdict converts 4 of 4 with its pre-fix contrast. The full-book path reaches ready in about 88 s.

Two refinements stay validated but unshipped, carried from Phase I: the split-conformal interval correction and the single-arm CUSUM posture. Neither enters the delivered behaviour, and both return in Chapter 7 as numbered future work. Transfer holds in the sense the design needs. Dense and hybrid reproduce their baselines exactly on the rerun, which validates the rerank arm. The DDIA corpus repeats the ACCA direction at larger magnitude. The limits are stated plainly. The scope is a single learner with synthetic histories in place of a classroom study. One difficulty

question is eligible, with the four replay caveats of Section 6.4. The loop that stayed dashed in Phase I runs solid here. Figure 4.2 shows where each stage from Table 4.1 now runs.

Chapter 7

Conclusion and Future Work

This chapter closes the report. Section 7.1 concludes what the two phases delivered. Section 7.2 sets out future work.

7.1 Conclusion

Phase I built an open-loop adaptive planner for a single self-directed learner. It estimated pace with Bayesian calibration and enriched shrinkage, and it flagged shifts with CUSUM. It projected completion with a Gaussian process and a cold-start composite, and it regenerated the plan with capacity bookings. Evaluation reported honest nulls with the wins, and the conformal correction and the single-arm CUSUM posture stayed validated but unshipped.

Phase II closed the loop with assessment-based verification of genuine learning. The principal novelty is the verified-learning closed loop, which extends the predict-then-optimize architecture [8] from logged time to graded evidence. Logged time stays gameable [11], so the plan responds to graded evidence.

Table 6.1 shows retrieval gains on both corpora with ranking parity on the dense baseline. Reranking lifts hybrid recall at 1 from 0.60 to 0.67 on the 754-chunk ACCA split and from 0.47 to 0.63 on the DDIA corpus. MRR rises by 0.077 on ACCA and by 0.139 on DDIA, while the live endpoint reads 0.813 reranked. Title-prefix indexing shows no gain, and int8 quantisation collapses to 0.10. The retired 788-split headline stays as frozen-split precedent only.

Grounded generation reads 0.8667 on the judge against 0.7803 gold support, with schema validity at 0.9611 and citation validity at 0.9133. The judge scores whether the cited chunk supports the answer, while gold support scores whether the citation contains the gold string. The coding arm refuses material it cannot ground, so derivability on the golds reads 0.0 as the honest terminal. A fresh live pull derives at 0.5349 and passes the 0.5 gate.

Tracing holds at parity on ranking while calibration nearly halves on live attempts. Bkt-v1 reads 0.7397 against 0.7368 for the running proportion over 282 observations, with ECE at 0.1401 against 0.2679. The shipped parameters hold mean AUC at 0.785 with ECE at 0.057, and the external anchor reads 0.602 to 0.611. Difficulty calibration stays unmeasured, with 23 observations on a single question in one band. The update stays as Equation 2.1 with the shipped filter in Listing 5.1.

Live verification confirms deployment only, with objective runs at 2 of 2, written grades at 4

of 4, and coding grades at 3 of 3. The full-book path reaches ready in about 88 s for the 572-page fixture, with zero console errors.

The loop verdict converts 4 of 4 `suggest_replan` verdicts into verdict-linked replans, as Table 6.4 records. Confidence reads 1.0 with sufficiency from 0.66 to 0.77, and every replan extends the deadline over a 32-day span. The pre-fix contrast is 4 of 4 hold at 0.18 to 0.34, and the gate moved from 0.9 to 0.7. Hook checks pass 37 of 37 and feedback checks pass 77 of 77.

Synthetic longitudinal replay reports next-week adherence at 0.000 with its rebound-era caveat, and the full 24-row emission sits in Appendix A. Capture ran one day before the deadline, so the zero reflects timing. The dip-window replay holds 4 of 4 at 0.28 to 0.35 in the trough and converts on the rebound eve at 1.0. Validated but unshipped are the split-conformal correction and the single-arm CUSUM posture.

7.2 Future Work

Table rows in Appendix A remain the reproducibility base for the work below. Four lines of work follow.

The first is to ship the validated refinements, in particular the split-conformal correction of the projection interval. The single-arm CUSUM posture ships with it, since detection feeds a human-confirmed prompt.

The second is longitudinal validation on real data, comparing closed-loop planning against the static baseline on adherence and on measured learning gain. A single learner over one term gives the honest next step beyond synthetic histories.

The third is difficulty calibration with two or more bands, so the adaptive chip moves from advisory to measured. It needs more eligible questions than the single band available here.

The fourth is classroom scope, extending the single-learner loop to groups without losing per-learner mastery.

Appendix A

Supplementary Tables and Figures

This appendix collects the supplementary tables and figures referenced from Chapter 6. Each item repeats its in-text number so the reader can move between the discussion and the full table without searching.

A.1 Adherence Emission

Table A.1 lists the full 24-row adherence emission from Section 6.4. Rates show actual over planned minutes for each user and week. Planned time reads 390 minutes for weeks 0 to 4 and 360 minutes for week 5. Week 5 reads 0.000 for all four users with the rebound-era caveat of Section 6.4.

Table A.1: Adherence emission: actual over planned minutes by user and week. Week 5 is the post-replan extension week.

User	w0	w1	w2	w3	w4	w5
steady	0.692	0.921	0.879	0.954	0.933	0.000
crammer	0.215	0.138	0.385	0.254	0.923	0.000
struggling	0.323	0.169	0.385	0.144	0.554	0.000
returning	0.656	0.000	0.000	0.700	0.900	0.000

A.2 Verdict Instants

Table A.2 gives the per-user verdict instants from Section 6.4. Each row joins its RoadmapReplanned by the exact verdict pair. All four converted. The 12 mastery rows are excluded here by design. They measure the seeded scripts at about 0.1067 throughout. The versioned JSON beside the replay evidence holds them.

Table A.2: Loop verdict instants: per-user decidedAt values.

User	Verdict	decidedAt	Converted
steady	suggest_replan	2026-10-14T12:00:11.235Z	yes
crammer	suggest_replan	2026-10-14T12:00:07.156Z	yes
struggling	suggest_replan	2026-10-14T12:00:07.606Z	yes
returning	suggest_replan	2026-10-14T12:00:07.515Z	yes

A.3 Second-Corpus Questions and Endpoint Pointers

Table A.3 lists the 30 DDIA gold questions from Section 6.1. The driver verified each snippet in at least one chunk before the run. Per-question ranks for the endpoint and harness arms stay in their versioned files:

- `research/doc/2026-10-22-endpoint-retrieval.json`
- `research/doc/2026-10-22-reranked-754.json`
- `research/doc/2026-10-22-second-corpus-ddia.json`

The DDIA gold stems sit in `research/doc/2026-10-22-ddia-questions.json` beside the run files.

Table A.3: DDIA second-corpus gold set, questions 1 to 15. Snippets are verbatim from the evidence file.

Q	Label	Question	Answer snippet
1	reliability meaning	What does it mean for a data system to be reliable?	The system should continue to work correctly (performing the correct function at the desired level of performance) even in the face of adversity
2	fault vs failure	What is the difference between a fault and a failure?	Note that a fault is not the same as a failure
3	scalability meaning	How does the book define scalability?	Scalability is the term we use to describe a system's ability to cope with increased load
4	maintainability operability	What does operability mean as an aspect of maintainability?	Make it easy for operations teams to keep the system running smoothly
5	document schema flexibility	What does schema flexibility mean in the document model?	Most document databases, and the JSON support in relational databases, do not enforce any schema on the data in documents
6	relational roots	What are the roots of the relational model?	proposed by Edgar Codd in 1970
7	NoSQL drivers	What drove the adoption of NoSQL databases?	need for greater scalability than relational databases can easily achieve
8	declarative queries	Why are declarative query languages preferred over imperative APIs?	declarative query languages like SQL turned out to be much better than imperative query APIs
9	SSTable merge	What is an advantage of SSTables over log segments with hash indexes?	Merging segments is simple and efficient, even if the files are bigger than the available memory
10	B-tree vs LSM reads	How do B-trees and LSM-trees compare on read performance?	LSM-trees are typically faster for writes, whereas B-trees are
11	secondary indexes	How common are secondary indexes in relational databases?	It is also very common to have secondary indexes
12	replication leaders	How does leader-based replication work?	One of the replicas is designated the leader
13	read-your-writes	What guarantee prevents reading stale data after your own write?	read-after-write consistency, also known as read-your-writes consistency
14	multi-leader conflicts	What is the biggest problem with multi-leader replication?	The biggest problem with multi-leader replication is that write conflicts can occur
15	quorum reads	When can quorum reads generally expect the most recent value?	If you have n replicas, and you choose w and r such that w + r > n

Table A.3: DDIA second-corpus gold set, questions 16 to 30.

Q	Label	Question	Answer snippet
16	skew batch	How does skew affect batch jobs?	due to skew (data not being spread evenly across worker processes) and needing to wait for the slowest task to complete
17	consistent hashing	What is consistent hashing?	Consistent hashing, as defined by Karger et al. [7], is a way of evenly distributing load
18	ACID coined	Where does the acronym ACID come from?	It was coined in 1983 by Theo Härder and Andreas Reuter
19	read committed	What is the most basic transaction isolation level?	The most basic level of transaction isolation is
20	snapshot isolation term	Which isolation level is named in the chapter on weak isolation?	snapshot isolation
21	serializable strongest	How strong is serializable isolation?	Serializable isolation is usually regarded as the strongest isolation level
22	2PC coordinator	What extra element appears in distributed transactions compared to single-node ones?	a coordinator (also known as transaction manager)
23	FLP result	What does the FLP result prove about consensus?	You may have heard about the FLP result
24	CAP consistency meaning	What does consistency mean in the CAP theorem?	the word consistency is used to mean
25	linearizability illusion	What illusion does linearizability give?	could give the illusion that there is only one replica
26	version vectors	What technique orders events across leaders?	a technique called version vectors can be used
27	batch definition	What is a batch processing system?	A batch processing system takes a large amount of input data, runs a job to
28	event vs processing time	What is the difference between event time and processing time?	Event time versus processing time
29	Kreps lambda	Who questioned the lambda architecture?	Jay Kreps: “Questioning the Lambda Architecture
30	exactly-once meaning	What does exactly-once mean for stream processing?	the final effect is the same as if no faults had occurred

Bibliography

[1] Chen, et al., "A Hierarchical Bayesian Model of Adaptive Teaching," *Cognitive Science*, 2024.

[2] Zanellati, A., et al., "Hybrid Models for Knowledge Tracing: A Systematic Review," *IEEE Transactions on Learning Technologies*, 2024.

[3] Mai, N. T., Cao, W., Liu, W., "Interpretable Knowledge Tracing via Transformer–Bayesian Hybrid Networks," *Applied Sciences*, doi:10.3390/app15179605, 2025.

[4] "Automatic Item Generation in Various STEM Subjects Using Large Language Model Prompting," *Computers and Education: Artificial Intelligence*, doi:10.1016/j.caeai.2024.100344, 2024.

[5] "Self-hosted Lecture-to-Quiz: Local LLM MCQ Generation with Deterministic Quality Control," arXiv:2603.08729, 2026.

[6] "Automated Generation and Tagging of Knowledge Components from Multiple-Choice Questions," doi:10.1145/3657604.3662030, 2024.

[7] "Automatic Grading of Short Answers Using Large Language Models in Software Engineering Courses," *Proc. EDUCON*, IEEE, doi:10.1109/educon60312.2024.10578839, 2024.

[8] Islam, M., et al., "Predicting Student Performance and Optimizing Study Schedule using Deep Learning and Dynamic Programming," *Proc. International Conference on Computer and Information Technology (ICCIT)*, IEEE, 2024.

[9] Alhazbi, S., et al., "Using Learning Analytics to Measure Self-Regulated Learning: A Systematic Review," *Journal of Computer Assisted Learning*, 2024.

[10] Uzun, Y., Suraworachet, W., Zhou, Q., "Engagement with Analytics Feedback and Its Relationship to Self-Regulated Learning Competence and Course Performance," *International Journal of Educational Technology in Higher Education*, doi:10.1186/s41239-025-00515-3, 2025.

[11] "Understanding Gaming the System by Analysing Self-Regulated Learning in Think-Aloud Protocols," doi:10.1145/3785022.3785025, 2026.